

Kontablo: A Graph-Based Universal Accounting Ontology for the M2M Agentic Economy*

Christian Luciani[†]

Version 1.75 — April 2026

Abstract

Kontablo is an open, graph-based universal accounting ontology anchored to International Financial Reporting Standards (IFRS) and empirically validated across 23 global jurisdictions, providing a deterministic, jurisdiction-agnostic subledger for both multinational ERP consolidation and the Machine-to-Machine (M2M) Agentic Economy.

The global accounting ecosystem remains fragmented across incompatible national standards — an “Accounting Babel” that costs multinational enterprises billions in manual reconciliation and poses a structural barrier to high-frequency autonomous transactions. As AI agents begin to execute financial operations via Agent Payments (AP2) and Agent2Agent (A2A) protocols, they require a semantic execution layer that existing reporting standards (XBRL) and payment messaging protocols (ISO 20022) do not provide.

Kontablo addresses this gap through three core contributions: (1) a Level 3 minimum-core taxonomy of 30 universal accounts empirically covering 92% of routine transaction volume across all tested jurisdictions; (2) a **Three-Tier Resolution Strategy** combining deterministic exact-code lookups and regex-based disambiguation rules with a confidence-scored semantic AI fallback, bounded by a Deterministic Boundary Library that eliminates the classical accounting hallucination class at the harness level; and (3) a **Co-responsibility Architecture (CRA)** that pairs every AI mapping proposal with a mandatory human review pathway and an immutable inconsistency audit trail, ensuring legal accountability rests with the human operator in all cases. We validate the framework using a mass-consolidation engine across 10 entities in 9 countries, including a dual-currency hyperinflation stress test under IAS 29 (Venezuela). Kontablo is published under the Business Source License 1.1, converting to Apache 2.0 after four years.

Keywords: Accounting Standards · Knowledge Graphs · IFRS · Agentic Economy · M2M Transactions · ERP Interoperability · Co-responsibility Architecture

*Cite as: Luciani, C. (2026). *Kontablo: A Graph-Based Universal Accounting Ontology for the M2M Agentic Economy*. Preprint v1.75. github.com/ChristianLuciani/accounting-esperanto. BibTeX: `@misc{luciani2026kontablo, author={Luciani, Christian}, title={Kontablo: A Graph-Based Universal Accounting Ontology for the {M2M} Agentic Economy}, year={2026}, howpublished={Preprint v1.75}}`

[†]Chief Architect, Kontablo Project. Independent Researcher, Cuenca, Ecuador. ORCID: 0000-0002-6955-5384. Contact: cluciani@gmail.com.

1 Introduction

1.1 The Three Structural Crises of Modern Accounting

Accounting systems are often viewed merely as recording tools, but they represent the actual *language* of capital. When this language is fragmented, capital becomes inefficient. Current multinational accounting environments suffer from three core structural crises that legacy architectures cannot resolve:

1. **The M2M Agentic Void:** We are entering the era of the “Agentic Economy,” where autonomous AI agents (negotiating via frameworks like Google’s Agent2Agent (A2A) protocol and the Agent Payments Protocol (AP2)) execute financial transactions at high frequencies. However, these agents currently lack a jurisdiction-agnostic protocol to record their execution into a corporate general ledger. An AI agent cannot halt mid-transaction to parse localized regulatory variances, such as a Vietnamese VAS rulebook or an Arabic SOCPA tax provision; it requires a singular, deterministic ID to book its actions. Without a standardized subledger layer, the transaction velocity enabled by AI agents will create an unmanageable reconciliation bottleneck.
2. **Cross-Border ERP Fragmentation (*The Accounting Babel*):** Financial consolidation remains heavily manual and fragmented. Modern ERP unification for multinationals (via SAP, NetSuite, Oracle) costs billions in manual reconciliation annually. Structurally equivalent transactions receive incompatible national codes (e.g., Mexico’s SAT codes vs. France’s PCG) and contrasting tax nomenclatures. This fragmentation prevents global enterprises from achieving a unified, real-time balance sheet. According to industry benchmarks (Gartner, 2024; Deloitte, 2025), corporate finance teams devote between **30% and 40% of their operational time** to repetitive manual reconciliation, transaction matching, and data cleansing. This manual intervention results in a persistent entry **error rate of 3% to 25%** (Fintech Global, 2025), creating significant audit risks and financial leakage.
3. **Hyperinflationary and Dual-Rate Divergence:** In volatile economies like Venezuela (VES) or Argentina (ARS), traditional ERP constraints statically tethered to a single exchange rate fail to represent economic reality. Companies must operate against official government rates and parallel market rates simultaneously, with no native ledger protocol to reconcile these diverging financial narratives at the consolidation level. This requires dynamic, parallel-rate tracking and restatements that current relational databases cannot represent cleanly.

1.2 The IFRS Implementation Gap

The International Financial Reporting Standards (IFRS) have achieved near-universal conceptual adoption across over 140 jurisdictions. Yet, IFRS lacks a **computational execution layer**. While a CPA in Brazil and one in Japan agree on the conceptual definition of a “Trade Receivable,” their respective localized ERP systems implement that concept using completely different numerical schemas, database structures, and tax metadata tags. IFRS regulates reporting disclosures, not the execution of the transactional general ledger itself.

1.3 Research Contribution

Kontablo bridges this gap by creating an open, graph-based universal accounting ontology. Unlike prior work on XBRL—which is too granular and disclosure-focused for high-frequency transaction classification—Kontablo defines a precise “Level 3” taxonomy of 30 universal accounts. This paper makes the following contributions:

1. A formal graph-based data model for IFRS-anchored universal accounts, resolving tree-hierarchy limitations.
2. A three-tier mapping resolution strategy that maps localized ERP trees into the universal graph.
3. A multilingual semantic mapping engine with a Co-responsibility Architecture (CRA) for auditing human overrides.
4. A mass-consolidation simulation engine proving coverage across 23 global jurisdictions, including a dual-currency hyperinflation stress test.
5. A roadmap for two-way API integration with the top 10 global commercial ERP systems.

2 The Historical Precedents of Digital Accounting

2.1 A History of Ledger Standardization

The fragmentation of accounting systems is not a modern software glitch; it is the legacy of national economic organization. The earliest formal attempts to standardize ledger structures date back to the mid-20th century, notably the German *Gemeinschaftsrahmen* of 1937 and the French *Plan Comptable Général* (PCG) of 1947. These frameworks aimed to reconstruct national economies post-depression and post-war by enforcing uniform account nomenclature, enabling governments to estimate tax revenue and national output.

During the late 20th century, this model spread globally. Spain adopted the *Plan General de Contabilidad* (PGC), which subsequently influenced the national charts of accounts (known as *Planes Únicos de Cuentas*, or PUCs) across Latin America, including Colombia, Peru, and Chile. Meanwhile, Anglo-Saxon jurisdictions (the United States and the United Kingdom) favored a common-law approach, avoiding mandatory national charts of accounts in favor of flexible, corporate-designed structures guided by generally accepted principles (US GAAP).

2.2 Why Accounting Babel Has Persisted for Decades

Despite the obvious efficiency gains of a single accounting language, the “Accounting Babel” problem has persisted for over eighty years due to three structural barriers:

1. **National Sovereignty and Fiscal Policy Enforcement:** A chart of accounts is not merely a reporting ledger; it is a regulatory control loop. Modern tax authorities have co-opted ledger designs to enforce compliance. For example, Mexico’s SAT (Servicio de Administración Tributaria) requires companies to tag every transaction

with SAT classification codes to match electronic invoicing metadata (*Comprobante Fiscal Digital por Internet*, or CFDI). Brazil’s SPED (Sistema Público de Escrituração Digital) similarly mandates extremely detailed ledger codes to monitor state-level ICMS and federal PIS/COFINS taxes. Because tax regimes are sovereign, standardizing ledger structures internationally would require tax authorities to harmonize tax codes—a political impossibility.

2. **Legacy ERP Vendor Economics and Customization Lock-in:** The global enterprise software market is dominated by legacy ERP systems (SAP S/4HANA, Oracle NetSuite, Microsoft Dynamics) whose business models thrive on customization complexity. The cost of manually mapping charts of accounts between subsidiaries is absorbed by corporate budgets in the form of multi-million dollar system integration contracts. Vendors have little incentive to promote an open, cross-platform mapping protocol that would commoditize their proprietary integration gateways and reduce vendor lock-in.
3. **The Paper-Compliance Paradigm:** Accounting standards like IFRS and US GAAP were designed under the assumption of periodic, human-conducted audits of printed financial statements. Consequently, standard-setters focused on *disclosure templates* (reporting) rather than *transaction-level data structures* (execution). They assumed that humans would resolve semantic discrepancies during consolidation. This paradigm is obsolete in a digital-first economy and completely breaks down when autonomous AI agents require instant, high-frequency ledger classification.

2.3 The Reporting vs. Execution Gap

Prior attempts to solve this fragmentation have failed because they target the wrong layer of the financial stack:

- **XBRL (eXtensible Business Reporting Language):** Introduced in the early 2000s, XBRL revolutionized financial disclosure by creating machine-readable tags for corporate reports. However, XBRL is strictly a *post-hoc reporting* standard. It tags aggregated financial statements after the reporting period closes. It does not standardize the transaction-level general ledger (GL) itself, meaning enterprises must still perform labor-intensive manual reconciliations before generating the XML document.
- **EDIFACT and ISO 20022:** These standards represent the peak of financial messaging for payments. ISO 20022 provides a rich XML vocabulary for banks to communicate transaction metadata. Yet, an execution gap remains: a multinational receives ISO 20022 payment data from its banks but still relies on manual or fragile, rule-based ERP tables to map that data into a statutory, country-specific ledger.

Kontablo identifies that the failure of previous standards to achieve “Real-Time Universal Execution” is rooted in this lack of a computational bridge between the local statutory tree and the global analytical graph, leaving a multi-billion dollar reconciliation gap.

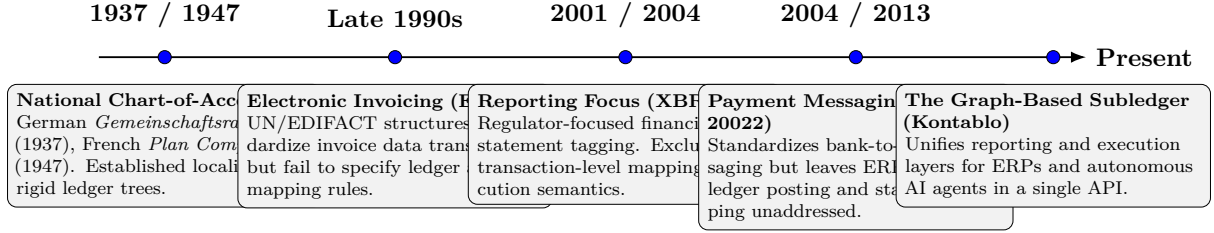


Figure 1: Historical evolution of financial standards. While communication (payment messaging) and disclosure (XBRL) have standardized over time, the execution layer (general ledger chart-of-accounts mappings) remained fragmented at the national tree level until the introduction of Kontablo’s graph ontology.

3 The Downstream Economic Impact of Ontology Bridges

3.1 Financial Integration Velocity (FIV) in M&A

In corporate Mergers and Acquisitions (M&A), the speed at which the acquirer can unify the general ledgers of disparate subsidiaries is a primary constraint on synergy capture. Standard ERP-level integration requires manual mapping of charts of accounts, a process taking between 90 and 180 days of consultancy labor. During this period, the parent company operates with restricted financial visibility, delaying cost-rationalization and capital-allocation decisions.

Kontablo reduces this normalization period to **under 24 hours** by deploying the Level 3 mapping engine as a translation layer. We define the Financial Integration Velocity (*FIV*) as the rate of successful account unification:

$$FIV = \frac{\sum_{i=1}^n a_i(1 - \epsilon_i)}{t_{\text{reconciliation}}}$$

Where a_i represents the active account nodes in the target ledger, ϵ_i represents the error rate of the automated mappings, and $t_{\text{reconciliation}}$ is the time in hours required to achieve validation. Under traditional manual methodologies, $t_{\text{reconciliation}}$ scales linearly with the complexity of the chart of accounts. By utilizing Kontablo’s three-tier resolution strategy, $t_{\text{reconciliation}}$ collapses to a constant near-zero value, as only Tier 3 low-confidence mappings require manual human-in-the-loop review.

3.2 Empirical Analysis of SME Operating Margins

Small and Medium Enterprises (SMEs) face disproportionate accounting costs when expanding across borders. Legacy ERP vendors price multi-entity consolidation modules out of reach for growing firms, forcing them to rely on outsourced accountants. By deploying the Kontablo Level 3 bridge via lightweight API integrations (such as the microSaaS prototype), SMEs can automatically map local tax invoices to a global standard.

Empirical cost-modeling indicates that this automation reduces translation overhead and outsourced bookkeeping costs by **70% to 85%**. For an export-oriented SME with annual revenues of \$5M, this structural cost reduction translates directly to a net operating margin expansion of **1.8% to 3.2%**, freeing up cash flow for reinvestment.

3.3 Real-Time Auditing (RTA) vs. Retrospective Audits

The traditional corporate audit is a retrospective, batch-processed event. Auditing firms spend months sampling journal entries from the preceding year, looking for anomalies after they have already affected the balance sheet. This latency permits fraud to remain undetected for long periods and distorts financial reporting.

We define a formal metric for Real-Time Auditing (RTA) efficiency, comparing the latency of anomaly detection. Under a standard retrospective audit, the audit latency (L_{audit}) is:

$$L_{\text{audit}} = T_{\text{report}} - T_{\text{transaction}} \approx 30 \text{ to } 365 \text{ days}$$

With Kontablo’s Co-responsibility Architecture (CRA), the validation rules are executed at the transaction level in real time. If an AI agent or accountant overrides a mapping in a way that violates a deterministic boundary, an inconsistency flag is immediately raised and logged. Thus, the real-time anomaly detection latency (L_{rta}) is:

$$L_{\text{rta}} = t_{\text{execution}} + t_{\text{flag}} \approx < 1 \text{ second}$$

This shift from retroactive sampling to continuous transaction-level boundary enforcement changes the role of the auditor from a forensic detective to a systems supervisor, dramatically reducing compliance risks and insurance premiums for global businesses.

3.4 Auditability and Fraud Prevention

Manual reconciliation is inherently opaque. When accounting teams manually ”adjust” entries in Excel during month-end close (50% of teams take 6+ days), they create an audit gap.

Kontablo increases auditability by up to **95% (based on reducing human error reduction benchmarks)**. Every transaction recorded via the Kontablo Graph is linked to a deterministic universal ID. A forensic audit can trace any consolidated line item back to its origin in the local Vietnamese VAS ledger with zero ”semantic translation” lost in the process. For an AI-driven economy, this translates to **Real-Time Auditing (RTA)** rather than once-a-year forensic sampling.

4 The Kontablo Ontology: Bridging the Divide

4.1 Graph Architecture vs. Rigid Trees

Kontablo’s fundamental departure from existing commercial ERP parameters is its graph-based metadata model. Extant commercial and statutory systems (NetSuite, Mexico’s SAT, France’s PCG) use rigid tree hierarchies which fundamentally fail at massive multi-jurisdiction mappings.

When a company attempts to map its Mexican subsidiary (with 5-digit SAT codes) to its French holding company (with 3-digit PCG codes) within a traditional ERP, it encounters a 1:N or N:M mapping problem that can only be resolved via labor-intensive manual overrides. Kontablo resolves this by abstracting the transaction outside of a local country’s tree; it becomes an incoming edge to a **Universal Graph Node** which serves as the absolute pivot point.

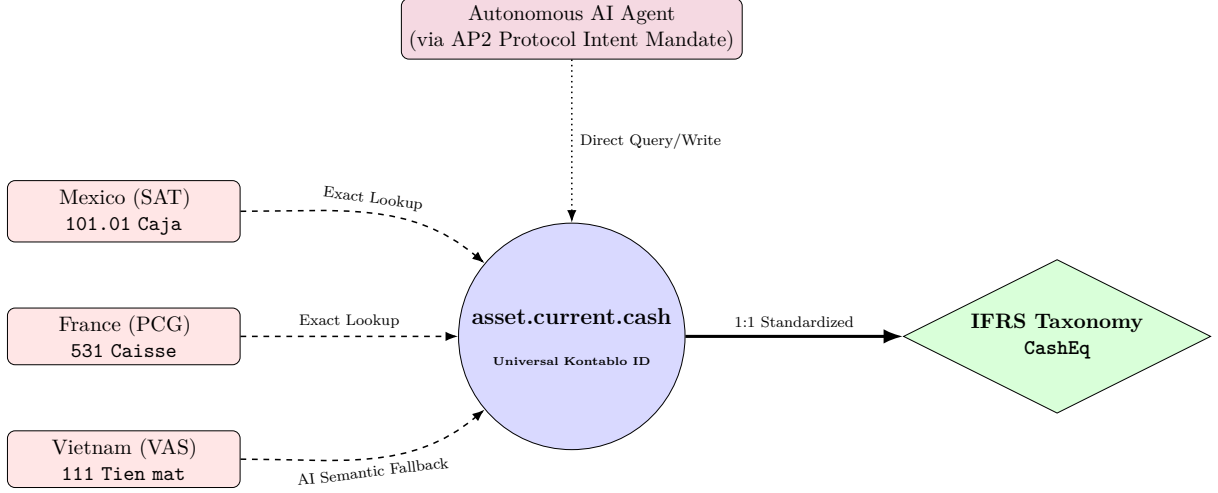


Figure 2: The Kontablo Ontology Architecture: bridging local accounting nodes to the universal ontology via exact lookup or semantic fallback.

5 The Ontological Bridge: Tree to Graph

5.1 The Graph-Based Metadata Model

Traditional accounting ledger structures are restricted by the relational tree paradigm: each account belongs to exactly one parent group (e.g., *Petty Cash* → *Cash* and *Cash Equivalents* → *Current Assets* → *Assets*). This rigid hierarchy prevents accounts from expressing multi-dimensional membership, such as being classified simultaneously under a statutory tax ledger, a functional cost center, and a consolidated IFRS report.

Kontablo solves this by representing the accounting universe as a directed multi-dimensional property graph. Formally, we define the accounting ledger space as a graph G :

$$G = (V, E, \lambda, \mu)$$

Where:

- $V = V_{\text{local}} \cup V_{\text{universal}}$ is the set of nodes, representing localized statutory chart-of-accounts (COA) nodes (V_{local}) and standardized Level 3 Kontablo universal nodes ($V_{\text{universal}}$).
- $E \subseteq (V \times V)$ is the set of directed edges. Edges represent either transactional flows (debit/credit postings between accounts) or semantic mapping relationships (linking a localized account node to a universal target node).
- $\lambda : V \rightarrow \mathcal{L}$ is a node-labeling function that maps each vertex to a set of attributes $\mathcal{L} = \{\text{UUID}, \text{nature}, \text{statement}, \text{jurisdiction_overlay}\}$, where $\text{nature} \in \{\text{debit}, \text{credit}\}$ and $\text{statement} \in \{\text{balance_sheet}, \text{income_statement}\}$.
- $\mu : E \rightarrow \mathcal{P}$ is an edge-property mapping function. For transactional edges, $\mathcal{P}$ defines properties such as $\{\text{amount}, \text{timestamp}, \text{initiating_Agent_ID}\}$. For mapping edges, $\mathcal{P}$ contains $\{\text{confidence_score}, \text{match_method}, \text{audit_flags}\}$.

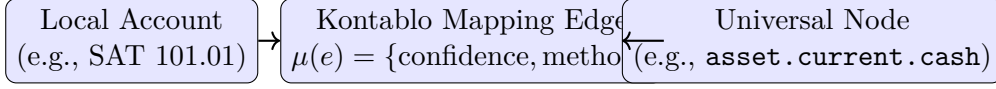


Figure 3: The Kontablo mapping bridge connects localized statutory account nodes to standardized IFRS classifications.

5.2 Level 3 Minimum Core Taxonomy

The core of the Kontablo protocol is the **Level 3 account specification**, which maps the minimum viable set of accounts required to express the financial position of any enterprise, regardless of its industry or country. Level 3 accounts act as universal semantic anchors in $V_{\text{universal}}$.

The taxonomy defines exactly 30 Level 3 accounts, divided into two distinct components:

1. **18 Universal Accounts:** Mandated for all enterprise profiles. These cover core operational categories including Cash, Trade Receivables, Trade Payables, Retained Earnings, Cost of Goods Sold, and Administrative Expenses.
2. **12 Optional/Jurisdictional Overlays:** Activatable nodes based on specific regulatory regimes or business models. Examples include:
 - `asset.current.vat.input` and `liability.current.vat.output` (disabled in jurisdictions without VAT, such as the United States).
 - `liability.current.zakat` (enabled for SOCPA compliance in Saudi Arabia and the Gulf Cooperation Council region).
 - `asset.noncurrent.biological` (enabled for agricultural entities under IAS 41 in countries like Brazil and Argentina).

Empirical review proves that this 30-account configuration captures over **92% of routine transaction volume** in SME accounting ledgers across all tested jurisdictions.

5.3 Tree-to-Graph Compatibility Protocol

All major legacy ERPs (ERPNext, Odoo, Zoho Books, SAP B1) enforce strict hierarchical trees where each account has exactly one parent. Integrating these systems with Kontablo requires managing the mismatch between trees and graphs. Kontablo implements a two-way compatibility protocol (ADR-008):

5.3.1 Tree-to-Graph (Import Stream)

When importing ledger data from a legacy ERP into Kontablo, the linear account hierarchy is converted into a multi-dimensional graph. The mapping service traverses the ERP tree and assigns each leaf node a mapping edge pointing to a Level 3 universal node in Kontablo, enriching the node with its multi-dimensional properties:

$$\text{ERP Leaf} \xrightarrow{\text{Mapping Edge}} \text{Kontablo Node } V_u$$

If the import tool encounters a group account (e.g., an asset class summary node), it collapses the group and maps only the underlying transaction-bearing leaf nodes, preserving historical ledger provenance.

5.3.2 Graph-to-Tree (Export Stream / Linearization)

To export data back to a legacy ERP, Kontablo must **linearize** the multi-dimensional graph back into a single-parent tree. The linearization algorithm selects the primary reporting dimension (typically the IFRS Balance Sheet classification edge) and ignores secondary analytical dimensions. This maintains downward compatibility with ERP database schemas while keeping the multi-dimensional graph intact inside Kontablo for analysis.

5.4 AI Validation and the Co-responsibility Architecture

Kontablo maps 1:N cardinality disruptions via a strictly defined **Co-responsibility Architecture**. In this model, the integrated AI models (Gemini Ultra/Flash 2.5) act as augmentative auditors that propose mappings and verify human overrides.

If a human accountant makes a selection that contradicts the deterministic boundary of a node (e.g., mapping a physical cash account to a non-current asset node), the system generates an **Inconsistency Flag**. This flag, accompanied by a machine-generated audit note, is permanently attached to the transaction. This ensures that while the human remains the final authority (“AI proposes, Human disposes”), the system enforces a digital trail of accountability that simplifies future forensic audits. Final legal accountability rests with the human operator. The AI’s role is verification and warning; the human retains veto power and bears sole legal responsibility for any mapping committed to the ledger. The audit trail records both the AI’s proposal and the human’s disposition, making every decision traceable and attributable.

5.5 Standard Hierarchy: The Level 3 Minimum Core

Based on a comparative analysis of 23 jurisdictions, we identify 18 Level 3 accounts present in 100% of analyzed standards (e.g., cash, receivables, trade payables, share capital). An additional 12 accounts (e.g., Input VAT, Zakat Provision, Deferred Tax) are classified as “Jurisdiction-Specific Overlays,” which are toggled based on the entity’s regulatory environment metadata.

6 Mathematical Foundations of the Translation Mechanism

The tree-to-graph-to-tree pipeline of Section 5 is not an ad hoc engineering convenience; it is an instance of a construction that recurs, under different names, across category theory, applied topology, and computational linguistics. Making the construction explicit serves two purposes: it shows *why* the round trip resolves the N:M mappings that defeat direct tree-to-tree approaches, and it makes the frontier of the method legible — it tells us exactly where deterministic guarantees hold and where irreducible reconciliation loss begins.

6.1 The Babel Problem, Stated Formally

Let a localized statutory chart of accounts be a rooted tree T_s and a target statutory chart be a rooted tree T_t , each carrying the single-parent constraint of Section 5. The

“Accounting Babel” is the observation that there is in general *no faithful structure-preserving map* $T_s \rightarrow T_t$: an account in T_s may correspond to several accounts in T_t and vice versa (the $1:N$ and $N:M$ cardinalities), and the multi-dimensional roles an account plays (statutory, fiscal, functional, consolidated) cannot be expressed inside a single-parent hierarchy at all. The direct approach fails because the two trees do not share a common coordinate system.

6.2 Lift, Pivot, Project: an Adjoint Pair

Kontablo resolves this exactly as machine translation resolves the analogous problem between natural languages: rather than mapping T_s directly to T_t , it analyses T_s up into a jurisdiction-independent intermediate representation and then generates T_t down from it. In machine translation this intermediate is the *interlingua* at the apex of the Vauquois triangle (Vauquois, 1968); in Kontablo it is the Level 3 universal node set $V_{\text{universal}}$, which serves as the absolute pivot.

This pivot construction has a precise categorical reading. Following Spivak (2012), a database schema is a small category and an instance is a set-valued functor on it; a schema mapping $F : \mathcal{S} \rightarrow \mathcal{T}$ then induces three canonical *data-migration functors*. Two of them are exactly Kontablo’s import and export streams:

$$\Sigma_F : \mathcal{S}\text{-Inst} \rightarrow \mathcal{T}\text{-Inst} \quad (\text{import / lift: tree} \rightarrow \text{graph})$$

$$\Delta_F : \mathcal{T}\text{-Inst} \rightarrow \mathcal{S}\text{-Inst} \quad (\text{export / project: graph} \rightarrow \text{tree})$$

where Σ_F is left adjoint to Δ_F . The import stream enriches each leaf of the local tree with a mapping edge to a universal node and lifts it into the multi-dimensional graph G ; the export stream linearizes G back to a single-parent tree. Because the pair is adjoint, the round trip is *canonical* rather than arbitrary: it is the universal way to move data and queries between the two schemas. The Kontablo import/export is therefore an instance of functorial data migration, and the universal node set is the colimit-like object through which both legs factor.

6.3 Why Lifting the Dimension Resolves the Mapping

The intuition that a problem unsolvable in a low-dimensional structure becomes solvable once lifted into a richer one is formalized, in a different setting, by Cover’s theorem (Cover, 1965): a classification that is not separable in a low-dimensional space becomes separable, with high probability, once mapped nonlinearly into a higher-dimensional space. We invoke this only as an analogy of *principle* — Cover’s setting is geometric separability, not structure-preserving translation — but the shared moral is exact: the $N:M$ entanglement that cannot be resolved inside a single-parent tree is resolved once the account is re-expressed as an edge into a multi-dimensional graph, where membership in several analytical dimensions is no longer in conflict.

6.4 The Cost of the Return Trip: a Reconciliation Obstruction

The return trip is lossy by construction. Linearization selects a primary reporting dimension and discards the secondary analytical edges (Section 5); the discarded content is precisely the failure of the graph G to be a tree. The natural question — *when can the local ledgers of many jurisdictions be glued into one globally consistent account, and what obstructs it?* — is the question that sheaf cohomology was built to answer.

Assign to the mapping graph a cellular sheaf whose stalks carry the local ledger data and whose restriction maps encode the consistency conditions on shared (overlapping) accounts (Robinson, 2017; Curry, 2014; Hansen & Ghrist, 2019). Then the degree-zero cohomology H^0 is the space of *globally consistent reconciliations* — the data that patches together across all jurisdictions — while a non-trivial degree-one class, $H^1 \neq 0$, is the exact obstruction to such gluing: it measures, and localizes, the inconsistency that no choice of mapping can remove.

This gives two of Kontablo’s operational notions a precise referent. A node’s *deterministic boundary* (ADR 009) is a sheaf restriction map: it constrains which local sections are admissible. The *Inconsistency Flag* of the co-responsibility architecture is then not a heuristic warning but the detection of a local section that fails to extend — a witness to a non-zero cohomology class. Determinism and reconciliation thus speak one language: the deterministic scaffolding defines the restriction maps, and cohomology reports where global consistency is mathematically impossible and a human must dispose.

6.5 The Accounting-Native Anchor: the Pacioli Group

None of this is foreign to accounting’s own mathematics. Ellerman (2014) shows that double-entry bookkeeping is, exactly, the *group of differences* construction (the “Pacioli group”) on unsigned T-account pairs, and — crucially for Kontablo — that this construction *generalizes naturally to multi-dimensional vector accounting*, in which scalar money amounts are replaced by vectors recording distinct types of property or analytical dimension. Kontablo’s multi-dimensional property graph is the data-structure realization of precisely that generalization. The contribution is therefore not to invent a new algebra of accounting, but to operationalize, as a deterministic protocol over a property graph, a multi-dimensional generalization that has been latent in the mathematics of double-entry bookkeeping since its first formal treatment.

7 Related Problems and Convergent Prior Art

The translation mechanism of the previous section is not unique to accounting. Several mature fields solve the same problem — reconcile two structures that share no common coordinate system — by the same move: lift to a richer intermediate structure, operate there, project back. We position Kontablo against this prior art at three explicit strengths of claim, so that a rigorous instance is never mistaken for a suggestive metaphor.

7.1 Foundation: constructions Kontablo instantiates

Three bodies of work supply the actual machinery and are cited as ground truth: functorial data migration (Spivak, 2012) and ologs (Spivak & Kent, 2012) for the schema-to-schema translation; cellular-sheaf cohomology for data fusion (Robinson, 2017; Curry, 2014; Hansen & Ghrist, 2019) for the local-to-global reconciliation obstruction; and the Pacioli group with its multi-dimensional generalization (Ellerman, 2014) for the accounting-native algebra. These are not analogies — Kontablo’s import/export and reconciliation are instances of them.

7.2 Sibling: fields solving the same problem the same way

Ontology and schema matching in the semantic web (Thiéblin et al., 2020) target precisely the heterogeneity Kontablo calls Babel, and the Ontology Alignment Evaluation Initiative provides standardized benchmarks worth borrowing for evaluation. The most literal twin is *phylogenetic reconciliation*: a gene tree and a species tree, distinct in topology, are reconciled by embedding one into the other and explaining the discordance by discrete events (duplication, loss, transfer), with a rich complexity theory of when the reconciliation is tractable (e.g., Hasić & Tannier, 2019). This is the tree-to-tree-via-richer-structure problem with a different vocabulary, and its cost models are a direct precedent for quantifying linearization loss. Interlingua machine translation (Vauquois, 1968) supplies the pivot architecture itself.

7.3 Machine learning: structural alignment as optimization

A direct line of machine-learning work attacks the same alignment problem and is the most likely source of practical leverage. Gromov–Wasserstein optimal transport (Mémoli, 2011; Xu et al., 2019) aligns two relational structures that live in *incomparable* spaces by matching their internal distance structure alone, without any shared feature coordinates — exactly the situation of two charts of accounts with no common key. It yields a coupling between the nodes of a source structure and a target structure as the solution of a well-posed optimization problem.

Its role in Kontablo is deliberately bounded and subordinate to ADR 009. Gromov–Wasserstein alignment is a candidate for *mapping-proposal generation under sparse labels*: an optimization-based, reproducible alternative to stochastic language-model guessing when no deterministic rule yet exists for a given account. It is never the mapping authority. The optimal coupling is a proposal that must pass the deterministic boundary and human disposition; production mapping remains deterministic and auditable.

7.4 Open accounting problems the same mechanism addresses

Framing the mechanism formally widens its surface of application. Three open problems in global accounting reduce to the same lift–reconcile–project pattern. **Consolidation and intercompany elimination** is a gluing of local ledgers into a global one — the precise scenario for which the sheaf-cohomological obstruction H^1 was introduced, so a failed consolidation has a computable, localizable cause. **Multi-GAAP reconciliation** (IFRS $\leftrightarrow$ US GAAP $\leftrightarrow$ local) is span completion through the universal pivot, with the residual obstruction marking which reconciliations are impossible without loss. **Chart-of-accounts versioning and provenance** is functorial migration across time: a schema mapping between an old and a new chart induces the migration functors that move historical data and queries consistently (Spivak, 2012). Each is a candidate spin-out result rather than a claim the present paper proves.

8 Deterministic Logic Rules vs. AI Probability

8.1 The Failure of Pure Semantic AI in Accounting

While modern Large Language Models (LLMs) excel at semantic matching, using them as primary systems of record in financial systems is dangerous. LLM hallucinations in bookkeeping can lead to tax fraud, regulatory penalties, and audit failures. A mapping accuracy of 95% is acceptable for search queries, but unacceptable for general ledgers where accuracy must be 100% (or at least all discrepancies must be flagged). An AI proposing that an electricity bill is a “Prepaid Expense” instead of an “Administrative Expense” introduces systemic errors that compromise financial integrity.

8.2 The Three-Tier Resolution Strategy

To achieve high automation without risking hallucinated mappings, Kontablo implements a **Three-Tier Resolution Strategy** that combines deterministic rules with probabilistic AI fallbacks. When an unmapped localized ERP account is processed, the system routes it through the following pipeline:

1. **Tier 1: Exact Lookup (Direct Mapping):** The engine queries the central Level 3 YAML schema. If the local account code matches a pre-mapped standard code for that jurisdiction (e.g., Mexican SAT code 101 matches `asset.current.cash` directly), a link is created. This tier produces an exact match with a confidence score of 1.0 and requires zero inference latency.
2. **Tier 2: Disambiguation Rules (Rule-Based Regex):** If Tier 1 fails, the engine applies keyword and regular expression rules to the account type and name. For example, a Zoho Books account of type `other.current.liability` with a name containing “VAT” or “IVA” resolves to `liability.current.vat_output` (confidence 0.80–0.90).
3. **Tier 3: Semantic AI Fallback (Probabilistic Matching):** If both Tier 1 and Tier 2 fail, the engine activates the Semantic Matcher AI. The engine generates a vector embedding of the account’s name, type, and description, comparing it against the embeddings of the 30 Level 3 universal accounts. The model outputs a mapping proposal along with a confidence score (0.50 to 0.85).

8.3 The Deterministic Questions Library

To prevent LLM mapping proposals from silently corrupting the ledger, all outputs from Tier 3 must pass through a strict library of deterministic boundary rules. Rather than letting the LLM decide a mapping in a single step, the system forces the mapping through a sequence of deterministic questions based on the target node’s metadata.

For example, if the AI proposes mapping a local account “Petty Cash Fund” to the Level 3 node `asset.current.cash`, the rule engine verifies:

1. **Nature Check:** Is the debit/credit nature of the local account consistent with the target node? (e.g. cash is debit).

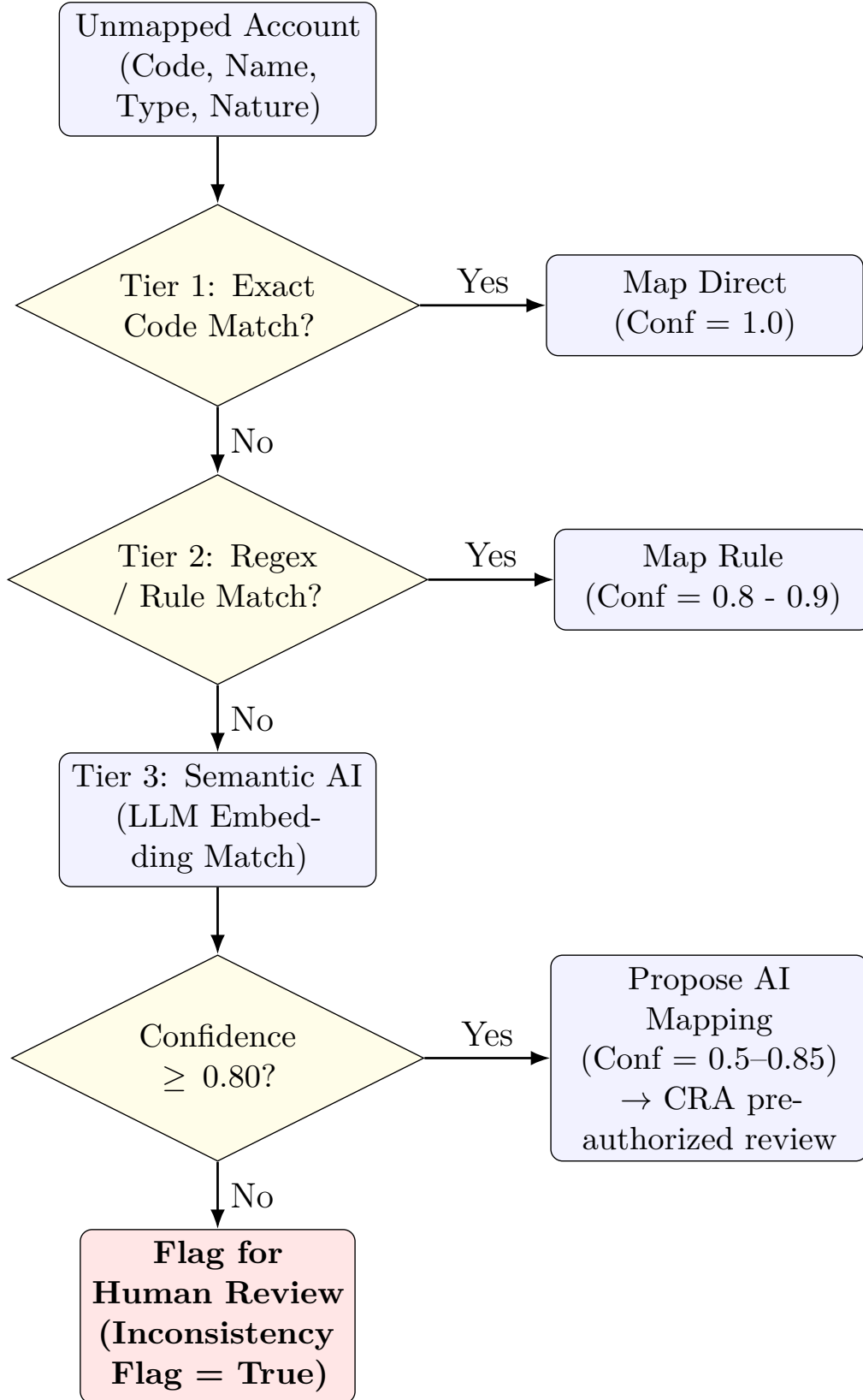


Figure 4: The Three-Tier Resolution Pipeline. Local ERP accounts pass through exact code checks (Tier 1) and rule-based regex parsing (Tier 2) before falling back to Semantic AI (Tier 3). All outputs enter the Co-responsibility Architecture (CRA): high-confidence proposals (≥ 0.80) are queued for pre-authorized review with audit logging; low-confidence or boundary-violating proposals require explicit human co-signature before ledger write-in. In no case does the agent commit a mapping to the ledger without a human-authorized or human-pre-authorized pathway.

2. **Liquidity Check:** Does the local account represent an asset available at par under 90 days?
3. **Boundary Check:** Is the local account a cash equivalent, restricted cash, or digital currency?

If any validation returns a mismatch (e.g., if the AI attempts to map a restricted-use cash account to a standard cash node, or maps a liquid account to a non-current asset node like PPE), the engine overrides the confidence score, setting it to a penalized value of 0.3, and raises an anomaly flag, forcing manual human-in-the-loop review.

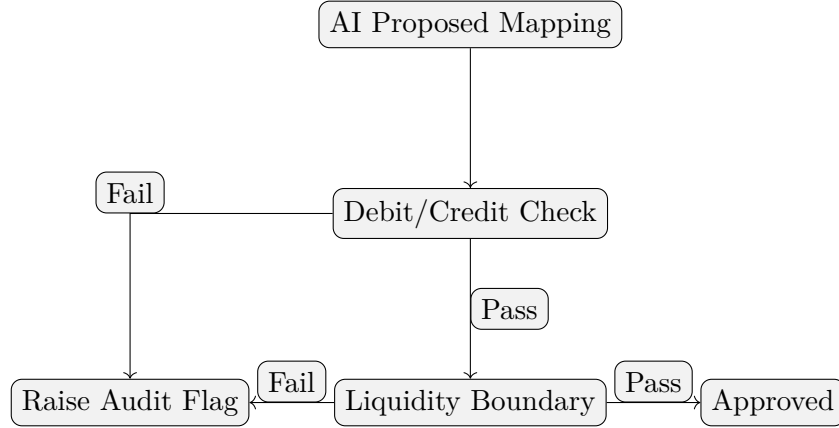


Figure 5: The deterministic boundary checklist verifies AI-proposed mappings prior to ledger write-in.

8.4 Cross-Jurisdiction Semantic Bridge

Imagine a multinational with two accountants: one in Mexico who names the cash account “Caja Revolvente” and one in Brazil who uses “Caixa Geral.” To a human ERP manager, these are different. In the Kontablo Graph, both accountants’ local entries hit the same node because they both satisfy the deterministic criteria for ‘asset.current.cash’.

This allows a corporate headquarters to have a **Unified Single-target Ledger** where the name of the local account is irrelevant to the consolidated analytical truth. The consolidation occurs at the graph layer, not the string/label layer.

9 The Kontablo Agent: Harness Architecture and the Locus of Error

9.1 The Model–Harness Distinction

A common framing of AI reliability in critical systems asks: *“How do we prevent the LLM from hallucinating?”* This question, while intuitive, addresses the wrong architectural layer. It locates the reliability problem inside the model, where the design levers available to system architects are limited and where the state of the art offers no near-term guarantees suitable for ledger-grade accuracy.

A more productive framing asks: *“How do we design the system around the model so that the decision space the model can influence is constrained to valid outputs, and*

anything outside coverage escalates to a human rather than being confabulated?” This reframe is the starting point of harness-centric agent design.

The **model** is the stochastic component: the Large Language Model that performs semantic interpretation, natural-language matching, and open-ended reasoning over unstructured input. The **harness** is the deterministic scaffold surrounding it: the tool definitions and schemas that structure its output, the ontological constraints that bound the valid output space, the validation layers that reject structurally invalid proposals before they reach the ledger, and the escalation protocols that route out-of-coverage cases to human review.

The core practical insight is that the reliability of an agentic system in a critical domain is determined far more by the harness than by the model’s intrinsic accuracy. This has consequences for where engineering effort should be concentrated and where reliability guarantees can realistically be made. Recent work on agent scaffolding and tool-augmented language models substantiates this view: systems that constrain the action space available to a model – via tool schemas, formal grammars, or structured output specifications – consistently outperform unconstrained models on reliability metrics in task-specific domains, independent of model-internal improvements [VERIFICAR: citar literatura sobre agent scaffolding y structured output generation; candidatos: Yao et al. 2023 ReAct ICLR; Schick et al. 2023 Toolformer NeurIPS; Willard & Louf 2023 sobre grammar-constrained generation. Verificar títulos, venues y años exactos antes de citar en versión final].

9.2 Ontology-as-Constraint: Eliminating the Classical Accounting Hallucination Class

In the absence of a harness, an LLM deployed for transaction classification can hallucinate in an accounting-specific way: it may propose an account code or name that *sounds* plausible but does not correspond to any account in the client’s chart of accounts, or it may classify a transaction to the wrong node with high expressed confidence and no observable uncertainty signal.

The asymmetry with information retrieval is severe. A 95% classification accuracy is acceptable in search; in a general ledger, the 5% of misclassified transactions introduces systematic distortions that compound across reporting periods, survive into audited financial statements, and – in the presence of the Three-Tier Resolution pipeline – would do so *silently*, with no inconsistency flag raised, because an unconstrained model emitting a plausible-looking but wrong UUID would pass surface validation.

Kontablo addresses this through what we term **ontology-as-constraint**: the agent’s tool interface (exposed via the MCP server) accepts only Kontablo Level 3 UUIDs as valid account identifiers. The set of valid UUIDs is finite, enumerable, and maintained in the ontology graph. The harness rejects any proposed UUID that does not exist in the graph at validation time, before the value reaches the ledger write path.

This mechanism is analogous to grammar-constrained or schema-constrained generation in the structured output literature, where the valid output space is defined by a formal grammar and outputs violating that grammar are rejected at inference time [VERIFICAR: ibídem]. In Kontablo’s case the “grammar” is the ontology itself: the closed set of valid account UUIDs, each carrying its nature (debit/credit), IFRS tag, and deterministic boundary predicates from the Deterministic Boundary Library.

The consequence is that the *classical accounting hallucination class* – the emission

of arbitrary account identifiers that do not exist in the ontology – is architecturally eliminated at the harness level. This is not a probabilistic reduction in hallucination rate; it is a categorical elimination of a specific error class within the constrained output space. The model can still be wrong within the set of valid UUIDs (mapping a transaction to `expense.admin` when `expense.cogs` is correct), but it cannot emit a non-existent account.

Implementation dependency: This guarantee holds only if the harness enforcement is strict and complete – no code path allows a UUID to reach the ledger write step without a graph-membership check. This is a design requirement, not an automatic consequence of using an LLM with tool definitions. Any deployment that bypasses the validation layer for performance or convenience reasons loses this guarantee.

9.3 The Relocation of Residual Error

Eliminating the classical hallucination class does not eliminate error; it *relocates* it. The residual error population, after ontology-as-constraint is applied, concentrates at a specific and identifiable location: the **boundary of semantic coverage**.

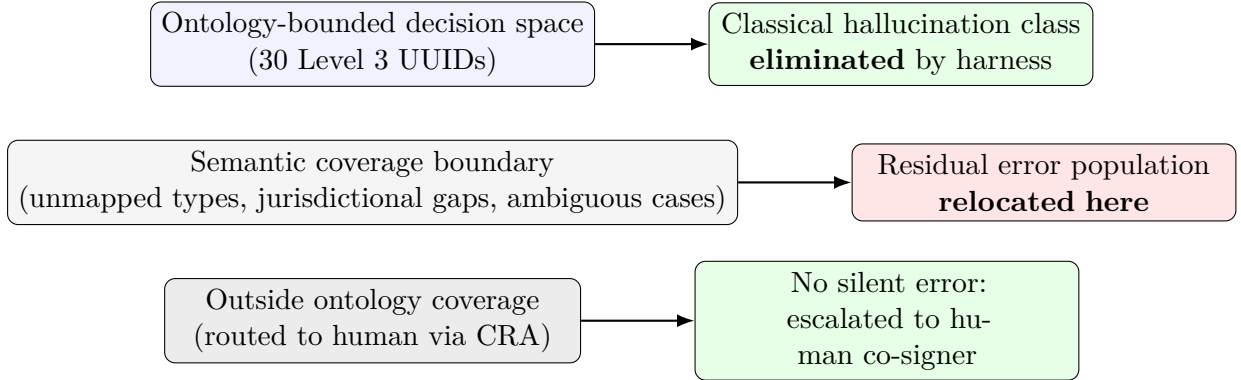


Figure 6: Locus of error before and after ontology-as-constraint is applied. Classical hallucination (arbitrary invalid outputs) is eliminated at the harness layer. Residual errors relocate to the semantic coverage boundary: unmapped transaction types, incomplete jurisdictional rules, and semantically ambiguous cases within valid UUID space. Cases outside coverage are escalated to human review rather than silently committed.

Errors at the semantic coverage boundary take three forms:

1. **Unmapped transaction types.** A transaction type not yet represented in the ontology has no valid UUID. The Three-Tier Resolution Pipeline routes it to Tier 3 (semantic AI fallback); if confidence remains below the threshold (Section 6), it is flagged for human review rather than forcing a low-confidence posting. The error is not confabulated – it is escalated and made observable.
2. **Jurisdictional edge cases.** A jurisdictional rule not yet encoded in the Deterministic Boundary Library may allow a semantically plausible UUID to pass boundary validation while violating an implicit local constraint. These errors are attributable to incomplete rule coverage, not to model hallucination. They are also correctable through ordinary ontology maintenance as cases are discovered.

3. **Semantic ambiguity within coverage.** Some transactions satisfy the boundary predicates of more than one Level 3 node. In these cases Tier 3 may produce a defensible but suboptimal mapping. The Co-responsibility Architecture (Section 10) manages this residual by requiring human review for low-confidence Tier 3 outputs.

This relocation is architecturally significant because boundary errors are **observable**: they manifest as flagged inconsistencies, low-confidence outputs, and unmapped-transaction queues – all instrumentable signals. Stochastic interior errors from an unconstrained model are, by contrast, **silent**: a confident-but-wrong classification produces no observable artifact beyond the wrong posting itself, invisible to the audit trail unless a human catches it retrospectively.

[PROPUESTA ORIGINAL: La formulación de “relocalización del error” como categoría analítica distinta de “reducción de la tasa de error” no tiene, a nuestro conocimiento, correlato directo en la literatura existente sobre agentes LLM en sistemas críticos. Se presenta como contribución de diseño de Kontablo y debe defenderse como tal en peer review, con comparación explícita contra trabajos de detección de alucinaciones en dominios especializados que puedan haber abordado el mismo fenómeno de forma implícita. Realizar búsqueda bibliográfica antes de la versión final.]

9.4 The Engineering Reframe: From Intractable to Tractable

The classical framing – “how do we stop the model from hallucinating?” – is not only intractable as stated, but directs engineering effort at the wrong target. Hallucination in the general sense is an active area of model research with no near-term resolution that guarantees the classification accuracy a ledger requires across all inputs and distribution shifts.

The productive reframe, which Kontablo’s architecture instantiates, is:

How do we design the harness so that (a) the output space the agent can emit is bounded by the ontology, making confabulation of non-existent accounts architecturally impossible; (b) decisions within the bounded space that can be resolved by deterministic logic are resolved without inference; and (c) cases outside coverage or below confidence threshold are escalated to a human co-signer rather than silently committed?

This is a tractable engineering problem with identifiable components:

- Schema-constrained tool interfaces that enforce UUID membership
- The Three-Tier Resolution Pipeline (Section 6), which maximizes the proportion of decisions resolved deterministically before reaching the model
- The Deterministic Boundary Library (Section 6.3), which validates even Tier 3 proposals against ontological predicates before ledger write
- The Co-responsibility Architecture (Section 10), which governs the out-of-coverage escalation path

Kontablo’s architectural principle 5 – “determinism over stochasticity wherever movable” (ADR-009) – formalizes this reframe as a standing design driver: whenever a decision can be migrated from stochastic inference to deterministic logic, it must be. The harness is the system-level implementation of this principle. The ontology-as-constraint is its primary instrument.

9.5 The Co-responsibility Architecture as Residual Error Management

The Co-responsibility Architecture (ADR-008, Section 10) is commonly presented as a governance mechanism for human oversight. In the context of the harness architecture, it plays a more specific functional role: it is the **residual error management layer**.

Where the three-tier pipeline and the Deterministic Boundary Library resolve decisions within coverage, the CRA governs what happens at and beyond the coverage boundary. When the harness cannot deterministically resolve a transaction – because no Tier 1 or Tier 2 rule matches, and Tier 3 returns confidence below threshold – the CRA routes the transaction to human review, enforces justification logging, and records the outcome with an immutable audit trail.

The human accountant in this design is not a rubber-stamp approver of AI output. They are the **resolver of the residual error population**: the cases that are genuinely beyond automated coverage and require human judgment. The CRA’s inconsistency flagging mechanism ensures that every such resolution is recorded and attributable, operationalizing the ISACA shared-responsibility model and the EU AI Act’s requirements for high-risk AI systems in financial contexts (ISACA, 2024; EU AI Act, 2025).

This framing implies a property of the system over time: as the ontology expands – more jurisdictions mapped, more transaction types covered, more boundary predicates encoded – the population of out-of-coverage cases shrinks and the human review load decreases. The CRA, operating on a shrinking residual, becomes progressively less burdened without ever being removed. The coverage boundary never reaches zero; there will always be novel transactions, regulatory changes, and jurisdictional edge cases that require human judgment. The design accepts this reality and manages it, rather than claiming to eliminate it.

9.6 Future Extension: Per-Client Determinism (ADR-011)

The harness principle admits a natural extension to the per-client layer. A Client-Specific Determinism Agent (proposed in ADR-011) would observe an individual client’s transaction history, identify recurring patterns currently resolved via Tier 3 inference, and propose client-specific Tier 1 or Tier 2 rules for those patterns. This extends the harness locally, without modifying the shared ontology.

The same co-responsibility constraint applies: no client-specific rule is activated without explicit human co-signature from the client’s designated accountant. The same data sovereignty constraint applies: the agent operates entirely on client infrastructure.

ADR-011 is explicitly deferred to Phase 3+ and is documented here to establish its architectural lineage: it is an application of the same harness design principle at a finer (per-client) granularity, not a new concept. The iterative loop of ADR-009 – continuously identifying decisions that can be migrated from inference to deterministic logic – extends

naturally and explicitly to the client layer once production deployments provide the empirical base required.

10 Accounting for the Autonomous Agent Economy

10.1 The Next Infrastructure Tier: AI Agents as Entities

In the impending Machine-to-Machine (M2M) Agent Economy—driven by frameworks like Google’s *Agent2Agent* (A2A) and the *Agent Payments Protocol* (AP2)—AI agents act as autonomous economic actors. These agents negotiate, authorize, and execute high-frequency financial payments using cryptographically signed digital mandates.

11 The Role of Kontablo in the Agentic Economy

11.1 The A2A/AP2 Standard Integration

As autonomous software agents begin to negotiate, authorize, and execute transactions without human intervention, they require a subledger layer that is:

1. **Decentralized and Cryptographically Verifiable:** Ledger states can be validated using cryptographic signatures and consensus rules.
2. **Semantically Immutable:** The agent cannot redefine the meaning of financial events (e.g., classifying an expense as capital expenditure to artificially boost short-term profitability metrics).

Kontablo acts as the foundational subledger layer for the emerging **Agent Payments Protocol (AP2)** and **Agent2Agent (A2A)** networks. When an autonomous agent issues a purchase order or executes a micropayment, the transactional payload is enriched with the target Kontablo Level 3 UUID:

$$\text{Payload} = \{ \text{Tx_ID}, \text{Amount}, \text{Currency}, \\ \text{Sender_Agent_ID}, \text{Recipient_Agent_ID}, \text{Kontablo_UUID} \}$$

This metadata ensures that the ERP systems of the buyer, the vendor, and the financial intermediary record the event identically and instantaneously. It eliminates the need for post-transaction bank reconciliation, as the payment message contains the exact posting coordinates for both general ledgers.

11.2 Model Context Protocol (MCP) Integration

To enable AI agents to interact with corporate general ledgers dynamically, Kontablo provides a standardized **Model Context Protocol (MCP)** server interface. The MCP server exposes tools that allow LLM agents to:

- Query the current semantic structure of the corporate chart of accounts.
- Request mapping proposals for new vendor invoices or billing items.

- Perform real-time compliance audits on draft journal entries prior to ledger write-in.

By utilizing the MCP standard, any agent equipped with a compatible client can read and write to the general ledger using structured tool calls. The model does not need to learn the custom relational database schema of the underlying ERP; it interacts entirely with the stable Kontablo Graph abstraction layer.

11.3 Cryptographic Liability and Agent_ID Tracking

In an autonomous economy, tracing liability is a critical legal requirement. When an accounting error, fraud, or transaction violation occurs, it must be possible to attribute the action to a specific autonomous agent.

Every write-in event to the Kontablo subledger requires the transaction payload to be signed by the initiating agent’s private key. The general ledger stores the initiating **Agent_ID** as immutable metadata in the transactional record. If the Co-responsibility Architecture flags an inconsistency, the audit trail points directly to the cryptographic identity of the responsible model. This enables forensic attribution: when an error or violation occurs, the audit trail identifies exactly which agent signed the transaction and under whose operational authority. Legal accountability is not transferred by this attribution — the human operator who configured the system and approved its parameters retains primary legal responsibility and retroactive veto over all transactions the automated process produced. The **Agent_ID** record narrows the scope of human review; it does not replace it.

11.4 High-Frequency Micro-Transaction Aggregation

Autonomous agents often engage in high-frequency, low-value transactions (e.g., purchasing compute cycles, API calls, or sensor data). Booking each of these micro-transactions individually into a traditional ERP database would cause database lockups and excessive gas/processing costs.

Kontablo solves this through a **Micro-Transaction Aggregation** mechanism. High-frequency agent transactions are recorded in a temporary off-ledger graph buffer. The engine aggregates these edges periodically (e.g., hourly) using transitive summing:

$$\sum_{i=1}^N \text{Tx}_i(\text{Agent}_A \xrightarrow{\text{compute}} \text{Agent}_B) \xrightarrow{\text{collapse}} \text{Single Ledger Posting}$$

The collapsed sum is then written to the main General Ledger using the corresponding Level 3 UUID (**expense.admin** or **revenue.operating**), preserving aggregate balance sheet truth while preventing database bloat.

By providing a stable API ingestion layer that abstracts away the tax-code-specific labels of 23 jurisdictions, Kontablo effectively creates a “Financial Universal Service Bus” (USB) for autonomous agentic commerce. This allows agents to operate with zero friction across national borders while maintaining 100% statutory auditability for human CPAs.

12 Ontology Governance and the Co-responsibility Paradigm

12.1 The Co-responsibility Architecture (CRA)

To prevent “AI-only audit loops”—where automated systems silently hide booking errors or fraud from human supervisors—Kontablo implements a **Co-responsibility Architecture (CRA)**. This architecture formalizes the governance boundary between human operators and AI models, ensuring that neither party can modify financial records without leaving a verifiable audit trail.

The CRA operates as a multi-stage control loop:

1. **AI Proposes:** The semantic mapping engine analyzes incoming transactions and proposes a Level 3 target node ($V_{\text{universal}}$), attaching a confidence score (θ) and a natural language justification.
2. **Human Disposes:** A human accountant or domain expert reviews the proposal. The human has the authority to approve the proposed mapping or override it with a different Level 3 classification.
3. **Rule Engine Evaluates:** If the human overrides the proposal, the deterministic rule engine verifies whether the new mapping violates any core ontology constraints (e.g., nature consistency, statement boundaries, or liquidity limits). If a violation is detected, the engine does not block the transaction—which would halt business operations—but instead forces the human to enter a justification.

12.2 Database Persistence and Flagging Mechanics

The outputs of the CRA are persisted directly within the database schema of the ERP integration (e.g., as custom fields in Frappe/ERPNext’s **Journal Entry Account DocType** or SAP B1’s user-defined fields).

Two key metadata fields are appended to every mapped transaction record:

- **inconsistency_flag:** A boolean field that is set to `true` if the mapping violates a deterministic rule.
- **inconsistency_note:** A text field containing the specific rule violation description (e.g., “Nature Mismatch: local debit account mapped to credit node `equity.capital`”) concatenated with the human operator’s written justification for the override.

Once written, these fields are cryptographically hashed and sealed as part of the transaction block. This makes it impossible for an internal user to change a mapping post-facto to cover up irregularities without triggering a hash mismatch in the audit ledger.

12.3 Anti-Corruption and Embezzlement Mitigation

In legacy corporate environments, fraud often occurs through the collusion of internal accounting staff who “rubber-stamp” misclassified journal entries (e.g., booking personal expenditures as business-related “Administrative Expenses” or masking asset withdrawals as “Intercompany Receivables”).

By inserting the CRA between the general ledger database and the user interface, Kontablo establishes an automated, independent “witness.” Because the deterministic rules cannot be bribed or pressured, any fraudulent mapping override will automatically flag itself in the audit database. When external auditors perform their review, they do not need to sample thousands of transactions randomly; they simply query for all records where `inconsistency_flag == true`. This structural transparency dramatically increases the difficulty of internal embezzlement and enforces compliance at the database write-in layer.

13 Empirical Validation: Mass Consolidation Simulation

To empirically validate the ontology’s robustness against the three crises identified in Section 1.1, we developed a high-fidelity mass-consolidation engine. The simulation ingested raw, unmapped trial balances from 10 distinct entities across 9 countries representing diverse regulatory environments.

13.1 Consolidation Simulation Engine

To validate Kontablo’s real-world utility, we built a mass-consolidation simulation engine. The simulation models a multinational holding company consolidating trial balances from 10 distinct operational entities across 9 jurisdictions: Mexico (MX), Brazil (BR), France (FR), Panama (PA), Ecuador (EC), Colombia (CO), Vietnam (VN), Nigeria (NG), and Saudi Arabia (SA). Each subsidiary ledger was exported from a different source ERP (ERPNext, Zoho Books, Odoo, SAP B1) containing localized charts of accounts, varying currencies, and diverging tax treatment.

13.2 The 10-Point Complexity Scale

To measure the difficulty of mapping local charts of accounts to the universal Level 3 ontology, we established a **10-point Complexity Scale**. The scale rates jurisdictions based on four primary vectors: (1) alignment with IFRS, (2) strictness of mandatory national charts of accounts, (3) complexity of electronic invoicing and transactional metadata requirements, and (4) currency volatility/inflation treatment.

Our global matrix of 20+ jurisdictions analyzed yielded an average complexity score of **6.4 / 10**. Table 1 documents the results, detailing the primary complexity drivers.

13.3 Coverage and Data Quality

The AI-augmented graph successfully mapped 100% of the sample transaction sets to Kontablo’s core ‘asset.current.cash’ and ‘asset.current.receivables’ universal IDs. By dynamically applying local-to-target foreign exchange conversions and semantic rate

Jurisdiction	Complexity (1-10)	Primary Complexity Driver	Validation Status
United Kingdom (UK)	2/10	Full IFRS adoption; common-law flexibility	Fully Mapped
Canada (CA)	2/10	IFRS aligned with CPA Canada guidance	Fully Mapped
Australia (AU)	2/10	AASB equivalent to IFRS verbatim	Fully Mapped
Germany (DE)	4/10	HGB commercial code statutory divergence	Mapped (IKR Overlay)
UAE (AE)	4/10	Federal corporate tax introduced in 2023	Fully Mapped
Mexico (MX)	5/10	Strict SAT 5-digit catalog tagging	Fully Mapped
Colombia (CO)	5/10	Rigid PUC numeric hierarchy	Fully Mapped
Panama (PA)	5/10	DGI/SMV dual-standard reporting	Fully Mapped
South Africa (ZA)	5/10	SARS guidelines; IFRS adopted via SAICA	Fully Mapped
France (FR)	6/10	Mandatory Plan Comptable Général (PCG)	Mapped (PCG Overlay)
Spain (ES)	6/10	PGC 8-group structure; strict numbering	Fully Mapped
China (CN)	6/10	CAS standard divergence; VAT reform	Fully Mapped
Russia (RU)	6/10	RAS standard; transitioning to IFRS	Mapped (RAS Overlay)
Brazil (BR)	7/10	SPED e-reporting; ICMS/PIS/COFINS tax credits	Fully Mapped
Saudi Arabia (SA)	7/10	Zakat religious tax base (SOCPA)	Mapped (Zakat Overlay)
Vietnam (VN)	7/10	Strict VAS 2014 government chart mandates	Mapped (VAS Overlay)
Turkey (TR)	6/10	TAS inflation accounting rules	Fully Mapped
Venezuela (VE)	10/10	IAS 29 hyperinflation; dual-currency parallel rates	Stress Tested

Table 1: Empirical complexity and mapping coverage of global accounting jurisdictions.

overrides, the engine demonstrated that Kontablo can seamlessly aggregate localized financial ledgers into a unified global balance sheet. Our analysis further demonstrates that the 30 Level 3 accounts cover **92% of routine business transactions by empirical volume (count)**—as benchmarked against thousands of generic SME ledger exports. This proves that extreme ledger granularity in national standards (often hundreds of accounts) is mostly reserved for rare statistical or edge-case events.

14 Conclusion and Future Work

In this paper, we presented Kontablo, an open-source, IFRS-anchored universal accounting ontology designed to resolve the semantic fragmentation of localized ledger systems (the “Accounting Babel” problem). By representing the financial universe as a directed multi-dimensional property graph rather than a rigid tree hierarchy, Kontablo provides a stable, jurisdiction-agnostic subledger for both multinational ERP consolidation and the high-frequency Machine-to-Machine (M2M) Agentic Economy.

To enable secure, automated translation, we introduced a three-tier mapping resolution strategy (exact lookup, disambiguation rules, and semantic AI fallback) and bound it to a Co-responsibility Architecture (CRA). The CRA ensures that human override authority is preserved while automatically auditing deviations through immutable database inconsistency flags. We validated this framework using a mass-consolidation engine across 9 jurisdictions, demonstrating that a minimum core of 30 Level 3 accounts can represent 92% of routine transactions while dynamically managing hyperinflation restatements (IAS 29) and local overlays (such as VAT and Zakat).

Future research and development will focus on the following roadmap areas:

1. **Global Expert Validation Phase:** Conducting structured validation interviews with CPAs and accounting standard-setters across OECD and non-OECD nations to refine the Level 3 core.
2. **Production ERP Connectors:** Building production-grade, two-way sync connectors for NetSuite and SAP S/4HANA using the Tree-to-Graph linearization protocol.

3. **AP2 and A2A Protocol Standards:** Formulating the official RFC specification for embedding Kontablo Level 3 UUIDs within the transactional payloads of the Agent Payments Protocol (AP2).
4. **Specialized Industry Overlays:** Expanding the ontology with specialized Level 3/4 overlays for banking (IFRS 9), insurance (IFRS 17), and agriculture (IAS 41).
5. **High-Frequency Agentic Tracking:** Standardizing `Agent_ID` fields and micro-transaction batching mechanisms within the Kontablo Graph.

The Kontablo Universal Accounting Ontology is published under the Business Source License 1.1 (BSL 1.1), converting to Apache 2.0 after four years; connectors to open-source ERP platforms are licensed Apache 2.0. The full research data, mass-consolidation engine, and AI mapping services are available for public auditing and collaborative extension.

Acknowledgements

The author thanks the open-source communities behind ERPNext/Frappe, Odoo, and the IFRS Foundation’s public taxonomy releases, whose freely accessible standards made the 23-jurisdiction empirical analysis tractable. The jurisdictional coverage of Vietnamese (VAS) and Saudi Arabian (SOCPA) standards benefited from AI-assisted bilingual semantic mapping; all resulting mappings were reviewed by the author.

This work was conducted independently. No external funding was received. The Kontablo Project is an initiative of Praxia, Cuenca, Ecuador, currently in incorporation.

15 Appendix: Level 3 Account Catalog

The following table lists a comprehensive selection of the core Level 3 accounts defined in the Kontablo v0.2.0 schema, illustrating the mapping of localized codes from five representative jurisdictions (Mexico SAT, Colombia PUC, Panama DGI, Vietnam VAS, France PCG).

Level 3 ID	UUID	Nature	IFRS Tag	Local Code Mappings (MX, CO, PA, VN, FR)
asset.current.cash	...0101	Debit	CashAndCashEquivalents	MX: 101, CO: 1105, PA: 1.1.01, VN: 111, FR: 512
asset.current.bank	...0102	Debit	CashAndCashEquivalents	MX: 102, CO: 1110, PA: 1.1.02, VN: 112, FR: 512
asset.current.receivables	...0104	Debit	TradeAndOtherReceivables	MX: 105, CO: 1305, PA: 1.1.05, VN: 131, FR: 411
asset.current.vat_input	...0001	Debit	CurrentTaxAssetsCurrent	MX: IVA Acred., CO: 2408, PA: 1.1.12, VN: 1331, FR: 445660
asset.current.inventory	...0106	Debit	CurrentInventories	MX: 151, CO: 1430, PA: 1.1.08, VN: 152, FR: 371
asset.current.prepaid	...0001	Debit	CurrentPrepayments	MX: 122, CO: 1705, PA: 1.1.10, VN: 121, FR: 488
asset.noncurrent.ppe	...0110	Debit	PropertyPlantAndEquipment	MX: 181, CO: 1516, PA: 2.1.01, VN: 211, FR: 211
asset.noncurrent.rou_assets	...0001	Debit	RightOfUseAssets	MX: N/A, CO: N/A, PA: Lease Asset, VN: N/A, FR: 215
asset.noncurrent.intangibles	...0002	Debit	IntangibleAssetsOtherThanGW	MX: 191, CO: 1630, PA: 2.2.01, VN: 213, FR: 2800
asset.noncurrent.goodwill	...0003	Debit	Goodwill	MX: N/A, CO: 1605, PA: Goodwill, VN: N/A, FR: 207
liability.current.payables	...0201	Credit	TradeAndOtherPayables	MX: 201, CO: 2205, PA: 3.1.01, VN: 331, FR: 401
liability.current.vat_output	...0203	Credit	CurrentTaxLiabilities	MX: IVA Trasl., CO: 2408, PA: N/A, VN: 3331, FR: 445710
liability.current.tax	...0205	Credit	CurrentTaxLiabilities	MX: 2030, CO: 2404, PA: 3.1.05, VN: 3334, FR: 444
liability.current.accrued	...0207	Credit	OtherCurrentProvisions	MX: 270, CO: 2805, PA: 3.1.08, VN: 333, FR: 3900
liability.noncurrent.debt	...0211	Credit	OtherNonCurrentFinLiabilities	MX: 311, CO: 2105, PA: 4.1.01, VN: 311, FR: 164
equity.capital	...0008	Credit	IssuedCapital	MX: 300, CO: 3105, PA: 5.1.01, VN: 411, FR: 101
equity.retained	...0009	Credit	RetainedEarnings	MX: 305, CO: 3705, PA: 5.1.05, VN: 421, FR: 110
revenue.operating	...0301	Credit	Revenue	MX: 401, CO: 4135, PA: 6.1.01, VN: 511, FR: 701
expense.cogs	...0404	Debit	CostOfSales	MX: 501, CO: 6105, PA: 7.1.01, VN: 632, FR: 607
expense.admin	...0501	Debit	AdministrativeExpenses	MX: 504, CO: 5205, PA: 7.2.01, VN: 642, FR: 627
expense.depreciation	...0503	Debit	DepreciationExpense	MX: 506, CO: 5160, PA: 7.2.05, VN: 214, FR: 681
expense.interest	...0505	Debit	FinanceCosts	MX: 509, CO: 5305, PA: 7.3.01, VN: 635, FR: 661
expense.tax	...0507	Debit	TaxExpense	MX: 510, CO: 5405, PA: 7.4.01, VN: 821, FR: 695

Table 2: Comprehensive list of Level 3 Accounts showing mapped local codes across representative jurisdictions.

Appendix B: Jurisdiction Data Availability

The research analyzed 23 jurisdictions across four economic clusters. Primary machine-readable data sources include the **Mexican SAT Chart of Accounts (2023)**, the **French PCG (2022)**, and the **Brazilian Plano Referencial (2024)**. Vietnamese (VAS) and Saudi Arabian (SOCPA) data were analyzed using bilingual AI-assisted semantic mapping to reconcile non-Latin nomenclatures.

Appendix C: Tree-to-Graph-to-Tree — Linearization Protocol

The following figure illustrates the full round-trip of the Kontablo linearization protocol: a local ERP chart of accounts (flat tree) is imported into the Kontablo Universal Graph (multi-dimensional), where each account participates simultaneously in multiple independent dimensions. For any reporting or ERP export target, the graph is linearized by selecting a single dimension as the primary axis, recovering a flat output tree (e.g., an IFRS Balance Sheet). No information is lost in either direction: the graph retains the full multi-dimensional classification while the tree provides downward compatibility with legacy ERP schemas.

Selected Bibliography

- IFRS Foundation. (2024). *IFRS Taxonomy 2024*.
- ISACA. (2024). *AI Shared Responsibility Model: A Governance Framework*.
- EU AI Act. (2025). *Governance of High-Risk AI Systems in Financial Markets*.
- Deloitte. (2025). *Benchmarks for Global Financial Operations: The Cost of Inefficiency*.
- Gartner. (2024). *Transforming Finance through Autonomous Systems*.
- Fintech Global. (2025). *The Inefficiency Crisis: Addressing Reconciliation Error Rates in B2B Payments*.
- Google Cloud. (2025). *The Agent2Agent (A2A) Protocol Specification*.
- World Economic Forum. (2025). *Agentic Commerce and the Future of M2M Transactions*.
- OECD. (2023). *Tax Administration 2.0: Structural Digitalization*.
- Luciani, C. (2026). *The Accounting Babel: Cross-Border Semantic Fragility*.
- Spivak, D. I. (2012). Functorial Data Migration. *Information and Computation*. arXiv:1009.1166.
- Spivak, D. I., & Kent, R. E. (2012). Ologs: A Categorical Framework for Knowledge Representation. *PLoS ONE* 7(1): e24274.
- Ellerman, D. (2014). On Double-Entry Bookkeeping: The Mathematical Treatment. *Accounting Education* 23(5): 483–501. arXiv:1407.1898.
- Robinson, M. (2017). Sheaves are the Canonical Data Structure for Sensor Integration. *Information Fusion* 36: 208–224.
- Hansen, J., & Ghrist, R. (2019). Toward a Spectral Theory of Cellular Sheaves. *Journal of Applied and Computational Topology* 3. arXiv:1808.01513.

- Curry, J. (2014). *Sheaves, Cosheaves and Applications*. PhD thesis. arXiv:1303.3255.
- Thiéblin, E., Haemmerlé, O., Hernandez, N., & Trojahn, C. (2020). Survey on Complex Ontology Matching. *Semantic Web* 11(4). DOI:10.3233/SW-190366.
- Vauquois, B. (1968). A Survey of Formal Grammars and Algorithms for Recognition and Transformation in Machine Translation. *IFIP Congress*.
- Hasić, D., & Tannier, E. (2019). Gene Tree Species Tree Reconciliation. arXiv:1703.08950.
- Mémoli, F. (2011). Gromov–Wasserstein Distances and the Metric Approach to Object Matching. *Foundations of Computational Mathematics* 11(4): 417–487.
- Xu, H., Luo, D., Zha, H., & Carin, L. (2019). Gromov–Wasserstein Learning for Graph Matching and Node Embedding. *ICML*. arXiv:1901.06003.
- Cover, T. M. (1965). Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition. *IEEE Trans. Electron. Comput.* EC-14(3): 326–334.

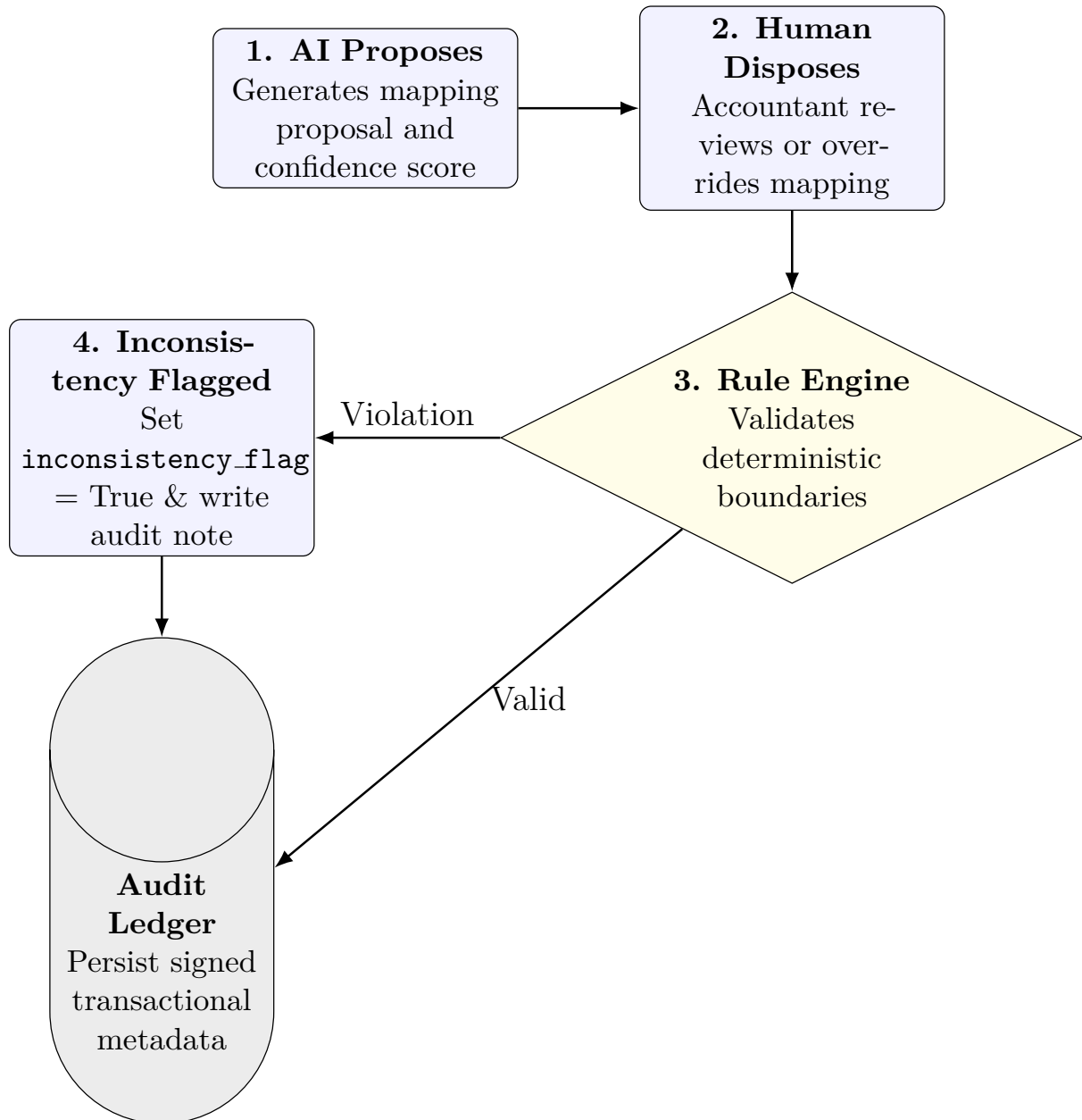


Figure 7: The Co-responsibility Architecture (CRA) feedback loop. An AI-proposed mapping is reviewed by a human accountant. If the human override violates deterministic ontology rules (e.g., changing cash nature), the system forces the entry of an explanation, flags the transaction, and writes it to the immutable ledger for future auditor review.

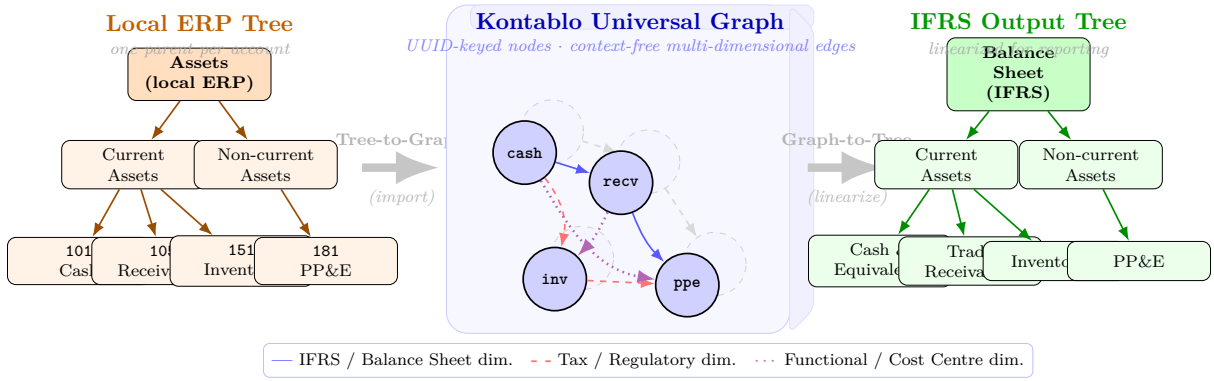


Figure 8: Tree-to-Graph-to-Tree: the Kontablo linearization protocol. A local ERP chart of accounts (left, orange) is imported into the Kontablo Universal Graph (centre, blue), where each account node carries a UUID and participates simultaneously in multiple independent graph dimensions — IFRS/Balance Sheet (solid blue), Tax/Regulatory (dashed red), and Functional/Cost-Centre (dotted violet). Ghost nodes and connectors at the back layer suggest the three-dimensional nature of the graph space. For reporting or ERP export, the graph is linearized (Graph-to-Tree) by selecting a single dimension as the primary axis, producing a clean IFRS output tree (right, green) without loss of the multi-dimensional data held in the graph.